Comparing SGD, SVRG, and SAGA for Regularized Logistic Regression

Jihyun Kim and William Zhang

1 Introduction

The goal of this project was to compare three stochastic optimization methods on the same convex machine learning problem. The methods were stochastic gradient descent (SGD), stochastic variance-reduced gradient (SVRG), and SAGA. All three methods were tested on L2-regularized logistic regression using the breast cancer dataset from `sklearn`.

This is a useful comparison because the three methods make different tradeoffs. SGD is simple and cheap per iteration, but its gradient estimates can stay noisy. SVRG reduces this noise by occasionally computing a full gradient at a snapshot point. SAGA also reduces variance, but it does this by storing past gradients for all samples. Because of this, the main question is not only which method reaches the lowest objective value, but also how much extra computation and memory that improvement costs.

The main result was that SVRG reached the lowest final objective value in the locked comparison, but it used about three times as many gradient evaluations as SGD. SAGA was competitive when tuned well, especially at learning rate 0.03, but at the locked learning rate 0.1 it was less stable than SVRG. SGD was the simplest and usually fastest method in wall-clock time, but its final loss was higher and its trajectory had more noise.

2 Problem Setup

The optimization problem was binary logistic regression with L2 regularization. For data points (x_i, y_i) with labels $y_i \in \{-1, +1\}$, the objective was

$$F(w, b) = \frac{1}{n} \sum_{i=1}^{n} \log \Big(1 + \exp \Big(-y_i(w^\top x_i + b)\Big)\Big) + \frac{\lambda}{2} \|w\|_2^2.$$

The bias term b was not regularized.

The dataset was `sklearn.datasets.load_breast_cancer`. It has 569 examples and 30 features. The labels were converted from the original `{0,1}` format to $\{-1, +1\}$. The features were standardized using `StandardScaler`. The class counts after conversion were 212 examples with label -1 and 357 examples with label $+1$.

All runs used

$$\lambda = 10^{-3}.$$

The starting point was the zero vector for both w and b. At this point the objective value was 0.69314718. I also checked the implementation of the gradient using finite differences. The maximum finite-difference error reported by the project code was 7.173×10^{-11}, and the mean of the per-sample gradients matched the full gradient up to numerical precision.

3 Methods

3.1 SGD

SGD used one randomly sampled data point per update. For a sampled index i_t, the update was

$$\theta_{t+1} = \theta_t - \eta \nabla f_{i_t}(\theta_t),$$

where $\theta = (w, b)$ and f_i is the single-example logistic loss plus the regularization contribution. SGD only needs the current parameter vector and one stochastic gradient, so it has the smallest memory cost. The drawback is that the gradient estimate can have high variance, especially near the optimum.

3.2 SVRG

SVRG keeps a snapshot point $\tilde{\theta}$ and computes the full gradient $\nabla F(\tilde{\theta})$ at that point. During the inner loop, it uses the variance-reduced estimator

$$v_t = \nabla f_{i_t}(\theta_t) - \nabla f_{i_t}(\tilde{\theta}) + \nabla F(\tilde{\theta}).$$

Then it updates $\theta_{t+1} = \theta_t - \eta v_t$.

In this project, the inner loop length was set to one epoch, meaning 569 inner updates per snapshot. This makes the method easy to compare against SGD, but it also means SVRG pays for a full pass over the data at the start of every epoch. So SVRG has low memory overhead, but higher gradient-evaluation cost.

3.3 SAGA

SAGA stores a table of past per-sample gradients. At each step it samples one index, computes a new gradient for that sample, and combines it with the stored table average. In the implementation, the gradient table stored the unregularized per-sample gradients, and the L2 gradient was added fresh at the current iterate. This avoids making the stored gradients depend on an old value of the regularization term.

SAGA has a lower gradient-evaluation count than SVRG in these experiments because it does one new stochastic gradient per update after initializing the gradient table. The main cost is memory: it stores one gradient vector for every training example. On this dataset that memory is still small, but it would grow linearly with both the number of examples and the number of features.

4 Experimental Setup

All methods used the same dataset, preprocessing, initialization, and objective. Each main run used 30 epochs. One epoch means 569 stochastic update steps for SGD and SAGA. For SVRG, one epoch means one snapshot full-gradient computation and 569 inner-loop updates.

The project tracked three main metrics:

- objective value,
- number of gradient evaluations,
- wall-clock runtime in seconds.

The learning-rate sweeps used seed 0. The tested learning rates were

$$\text{SGD: } \{0.001, 0.003, 0.01, 0.03, 0.1\},$$

$$\text{SVRG and SAGA: } \{0.005, 0.01, 0.03, 0.05, 0.1, 0.2\}.$$

After the sweep, the locked learning rates used in the main comparison were 0.03 for SGD, 0.1 for SVRG, and 0.1 for SAGA.

5 Results

5.1 Main Locked Run

Table 1 shows the final metrics for the main locked run with seed 0. All methods started from the same zero initialization and ran for 30 epochs.

Table 1: Main locked comparison, seed 0.

Method	Learning rate	Final loss	Gradient evals	Runtime (s)
SGD	0.03	0.0641138605	17070	0.1774
SVRG	0.10	0.0598532343	51210	0.3122
SAGA	0.10	0.0627392703	17639	0.2262

SVRG had the lowest final loss in this run. SGD was the fastest in wall-clock time, but it also ended with the largest loss. SAGA used almost the same number of gradient evaluations as SGD, plus the initial gradient-table pass, but its locked learning rate was not the best one from the sweep.

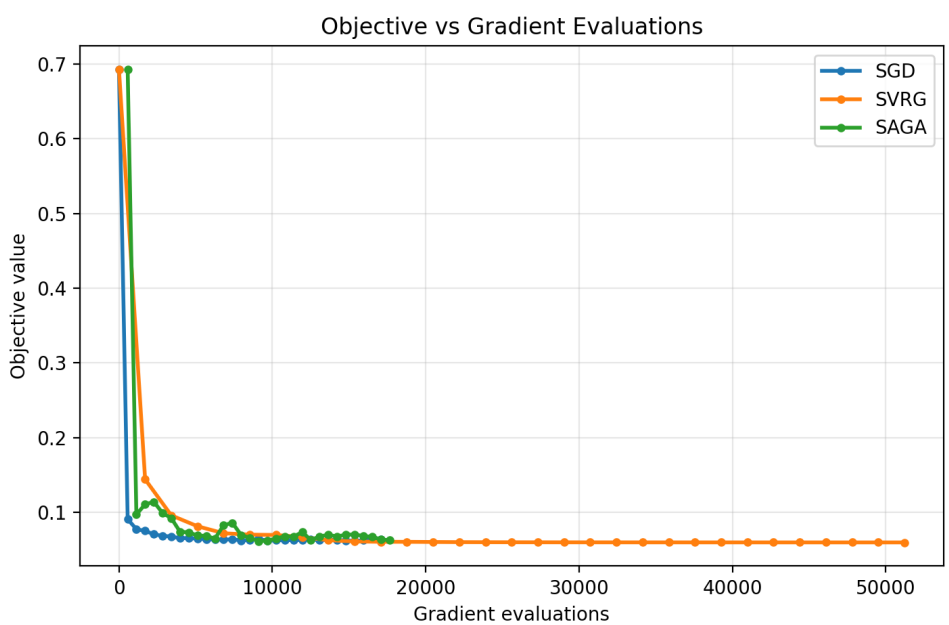

Figure 1: Objective value versus gradient evaluations for the locked run.

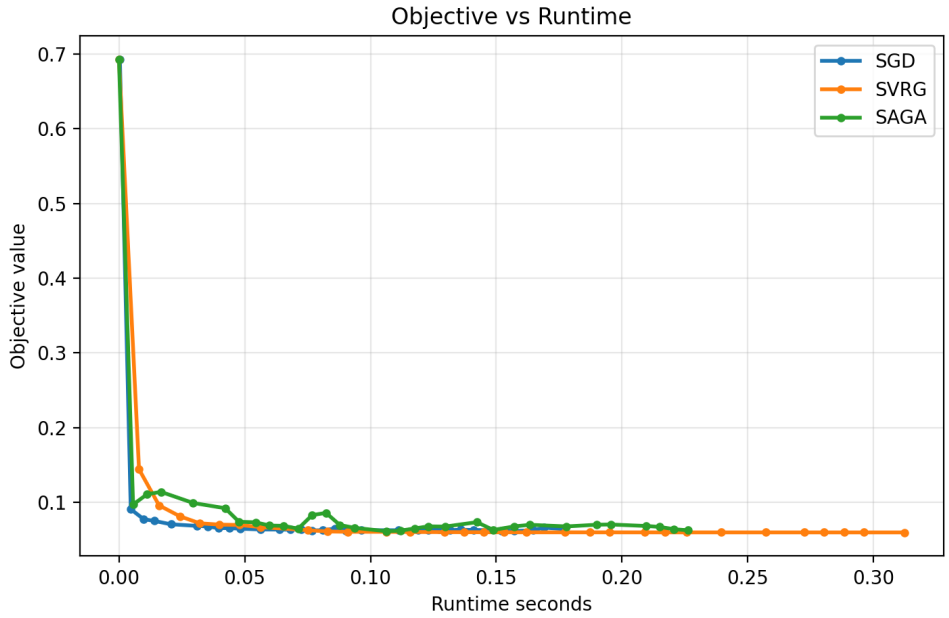

Figure 2: Objective value versus runtime for the locked run.

The gradient-evaluation plot shows the main cost difference clearly. SVRG makes smoother progress and reaches the best objective value, but it spends many more gradient evaluations because each epoch includes a full-gradient computation and two per-sample gradients per inner update. SGD gets a large decrease quickly, but then fluctuates around a higher objective value. SAGA also drops quickly, but the curve is less smooth at the locked step size.

5.2 Learning-Rate Sweep

Table 2 summarizes the best learning rate for each method from the sweep. The best value was chosen by final objective value after 30 epochs.

Table 2: Best learning rates from the sweep, seed 0.

Method	Best learning rate	Final loss	Gradient evals	Runtime (s)
SGD	0.03	0.0641138605	17070	0.2065
SVRG	0.10	0.0598532343	51210	0.3574
SAGA	0.03	0.0603328688	17639	0.2579

This table changes the interpretation of SAGA. At learning rate 0.1, SAGA ended at a final loss of 0.0627392703 in the locked run. In the sweep, its best learning rate was 0.03, where it ended at 0.0603328688. That is close to SVRG's final loss while using far fewer gradient evaluations.

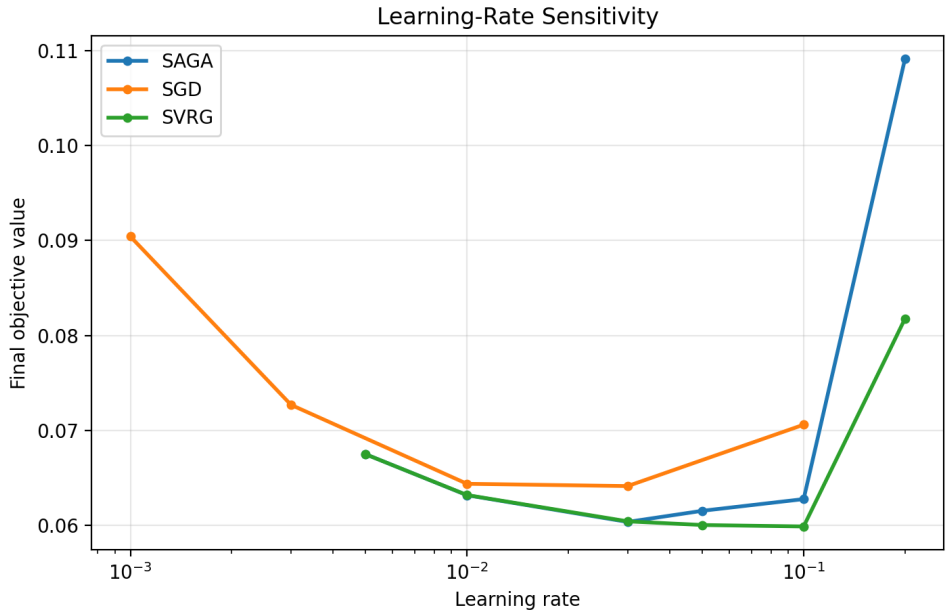

Figure 3: Final objective value after 30 epochs for each learning rate.

Table 3: Full learning-rate sweep, seed 0.

Method	Learning rate	Final loss	Gradient evals	Runtime (s)
SGD	0.001	0.0904283298	17070	0.2301
SGD	0.003	0.0726797067	17070	0.2304
SGD	0.010	0.0643535930	17070	0.1775
SGD	0.030	0.0641138605	17070	0.2065
SGD	0.100	0.0705829100	17070	0.1857
SVRG	0.005	0.0674639556	51210	0.5650
SVRG	0.010	0.0631672775	51210	0.4358
SVRG	0.030	0.0603956776	51210	0.3866
SVRG	0.050	0.0600083114	51210	0.3215
SVRG	0.100	0.0598532343	51210	0.3574
SVRG	0.200	0.0817626251	51210	0.3402
SAGA	0.005	0.0674698286	17639	0.2339
SAGA	0.010	0.0631498309	17639	0.2509
SAGA	0.030	0.0603328688	17639	0.2579
SAGA	0.050	0.0615052350	17639	0.2241
SAGA	0.100	0.0627392703	17639	0.2113
SAGA	0.200	0.1091551955	17639	0.2539

The sweep shows that all three methods are sensitive to the learning rate. Small learning rates were stable but slower. Very large learning rates were not always better. For SGD, 0.1 hurt the final loss compared with 0.03. For SVRG, 0.2 was too large and produced a worse result. For SAGA, 0.2 was clearly too large in this experiment.

5.3 Seed Check

The project output folder also includes a locked three-seed summary for SGD and SVRG using seeds 0, 1, and 2. Those results are shown in Table 4. The available seed summary file did not include a SAGA row, so I treat this table as a stability check for the two methods present in that file rather than as a full three-method seed study.

Table 4: Locked seed summary from seeds 0, 1, and 2.

Method	Learning rate	Mean final loss	Std. final loss	Mean runtime (s)	Mean grad evals
SGD	0.03	0.0625934339	0.0013355237	0.3879	17070
SVRG	0.10	0.0598533388	0.0000094993	0.6513	51210

The seed check supports the main observation that SVRG is more stable in final objective value. Its standard deviation across these three seeds was much smaller than SGD's. This makes sense because SVRG uses a variance-reduced gradient estimator, while SGD keeps using plain stochastic gradients.

6 Tradeoff Analysis

Table 5 summarizes the practical tradeoffs. The memory numbers are based on the active optimizer state in the implementation, using 64-bit floats, $n = 569$, $d = 30$, and $p = d + 1 = 31$ parameters including the bias. This does not count the dataset itself, since all methods use the same data matrix.

Table 5: Compute and memory tradeoffs for the locked setup.

Method	Final loss	Grad evals	Runtime (s)	State memory	Main tradeoff
SGD	0.0641138605	17070	0.1774	about 0.5 KB	cheapest, but noisy
SVRG	0.0598532343	51210	0.3122	about 1.2 KB	best loss, more compute
SAGA	0.0627392703	17639	0.2262	about 138.5 KB	more memory, fewer evals than SVRG

For this small dataset, the memory cost of SAGA is not a real problem. Its gradient table is only about 138.5 KB. But the scaling matters. If the dataset had millions of examples or a much larger feature dimension, SAGA's $O(nd)$ memory could become expensive. SGD and SVRG only need $O(d)$ optimizer state, so they are much easier to scale in memory.

In terms of convergence quality, SVRG was the strongest method in the locked comparison. It reached the lowest loss and had the smoothest final behavior. The cost is that it used 51210 gradient evaluations, compared with 17070 for SGD and 17639 for SAGA. So SVRG is attractive when gradient evaluations are cheap enough and we care about getting a slightly lower objective value.

SAGA is most worth its memory cost when the gradient table fits comfortably in memory and when a well-tuned learning rate is used. In the sweep, SAGA with learning rate 0.03 reached 0.0603328688, which was close to SVRG's 0.0598532343, while using about one third as many gradient evaluations. That is the strongest case for SAGA in this project. The weaker case is the locked learning rate 0.1, where SAGA was still reasonable but not as good as its tuned result.

SGD is still useful as a baseline. It was easy to implement, used the least memory, and had the fastest locked runtime. But it did not reach the same final objective value as the variance-reduced

methods. In this experiment, SGD is the best choice if simplicity and runtime are more important than squeezing out the lowest objective value.

7 Conclusion

This project compared SGD, SVRG, and SAGA on the same convex logistic regression problem. The experiment was intentionally kept simple: same dataset, same preprocessing, same initialization, same number of epochs, and the same objective for all methods.

The main takeaway is that variance reduction helped. SVRG reached the lowest final loss in the locked comparison and was also very stable across the seed check. Its downside was the extra compute from repeated full-gradient calculations. SAGA was competitive when tuned properly, especially at learning rate 0.03, where it came close to SVRG while using far fewer gradient evaluations. Its main downside is the gradient table memory, which is small here but can matter for larger datasets.

Overall, I would choose SGD for the simplest and fastest baseline, SVRG when I want the most reliable final objective value and can afford more gradient evaluations, and SAGA when the gradient table fits in memory and the learning rate has been tuned carefully.